PhaseKit: A Mean-Field Initialization Diagnostic with Finite CIFAR-10 Evidence and Explicit Input Moments

Anonymous

Preprint

Abstract

Deep networks without normalization or skip connections can be difficult to train when forward signals or backward gradients vanish or explode across depth. Mean-field theory describes this through the per-layer susceptibility χ_1, and the gradient depth scale derived from it, but practitioners rarely apply it to a concrete `nn.Module`. We present PhaseKit, a PyTorch tool that traces a model, propagates the mean-field second moment through its computation graph using the model's actual weight scales, and reports whether the total susceptibility χ_{tot} lies in the fixed window $[10^{-4}, 10^{4}]$. The default check assumes input second moment one, needs no dataset and takes a median of 0.10 s per network on a CPU. We test the prediction against measured outcomes: 1296 SGD training runs of 486 plain MLPs and CNNs on CIFAR-10, spanning three activations, depths from 4 to 40 and weight scales from 0.5 to 3.0. PhaseKit's binary prediction matches measured trainability with balanced accuracy 0.907 (95% CI 0.865–0.945); 99.0% of networks it calls trainable do train. Under the same protocol the classical $L \leq 6\xi$ depth-scale rule reaches 0.830 and a simple first/last-layer gradient-norm ratio probe of our own design, computed on one real minibatch, reaches 0.515. A control with the same $\ln 10^4$ window applied to the fixed-point depth scale reaches 0.913 balanced accuracy. Under the evaluated protocol, predicted phases are associated with different failure modes. Fresh independent-weight synthetic gradient diagnostics confirm that input-moment assumptions can change the phase; these are not training trials. We also evaluate seven external initialization schemes, including the public implementations of LSUV, GradInit, MetaInit and AutoInit, and find that PhaseKit's phase of each initialized network predicts its trainability across schemes (balanced accuracy 0.912 over 168 networks).

1 Introduction

Initialization can impair optimization in plain deep networks. Gradients vanish or explode exponentially with depth, the loss stays at its initial value or becomes non-finite, and the compute spent on the run is lost. Normalization layers and residual connections make modern architectures far less sensitive to this, but plain stacks of linear or convolutional layers remain common in scientific and tabular models, in theoretical studies, and as components of larger systems. For these networks, the question "will this initialization train?" has a classical theoretical answer. Mean-field analyses of random networks [11, 14] show that signal propagation is governed by the per-layer susceptibility $\chi_1 = \sigma_w^2\, \mathbb{E}[\phi'(\sqrt{q^*}z)^2]$: when $\chi_1 < 1$ gradients shrink geometrically with depth (the ordered phase), when $\chi_1 > 1$ they grow (the chaotic phase), and the depth scale $\xi = 1/|\ln \chi_1|$ bounds the depth that can be trained.

Applying this answer to a concrete model is usually done by hand, layer by layer, and ends with a judgement on how far from $\chi_1 = 1$ is too far for the given depth. PhaseKit automates the computation and the decision for a traced PyTorch `nn.Module`. This paper evaluates the one claim on which the tool's usefulness rests:

> *Under a stated input moment, the depth-calibrated mean-field phase computed from an untrained plain network predicts the selected finite-budget CIFAR-10 training label.*

We test the claim by training the networks. Our contributions are:

1. A data-free trainability check for `nn.Module` objects that propagates the mean-field second moment through the traced computation graph and thresholds the total susceptibility with a fixed window (§3, §4).
2. An evaluation against 1296 CIFAR-10 training runs of 486 plain MLPs and CNNs, with fixed labelling rules, three seeds per configuration and four reference predictors (§5.2).
3. A prediction evaluation across seven external initialization schemes on deep plain networks, running the public code of LSUV [8], GradInit [17], MetaInit [3] (as implemented by the GradInit authors) and AutoInit [2], in which the phase PhaseKit assigns to each initialized network is again scored against its training outcome (§5.3).

2 Background and Related Work

Signal propagation at initialization. For a fully connected network with weights $W_{ij}^{(l)} \sim \mathcal{N}(0, \sigma_w^2/n_{l-1})$ and activation ϕ, the pre-activation second moment obeys $q^{(l+1)} = \sigma_w^2 \, \mathbb{E}_z[\phi(\sqrt{q^{(l)}}z)^2] + \sigma_b^2$ in the wide limit, and the squared norm of a backpropagated gradient is multiplied by $\chi_1^{(l)} = \sigma_w^2 \, \mathbb{E}_z[\phi'(\sqrt{q^{(l)}}z)^2]$ per layer [11, 14]. Schoenholz et al. [14] showed empirically that networks train when depth is below a small multiple of the depth scale, approximately $L \leq 6\xi$. The theory has been extended to convolutional networks [15], residual networks [16] and to the full input-output Jacobian spectrum (dynamical isometry) [13, 10]; Hanin [5] characterized which architectures have exploding or vanishing gradients at finite width, and Roberts et al. [12] develop finite-width corrections systematically. PhaseKit uses the first-order theory and does not rely on these refinements.

Initialization schemes. Glorot (Xavier) initialization [4] and He (Kaiming) initialization [6] choose σ_w so that variances are preserved for linear and ReLU networks. LSUV [8] rescales an orthogonal initialization layer by layer until each layer's output has unit variance on a data batch. MetaInit [3] and GradInit [17] learn per-layer scale factors by gradient descent on an objective computed from data: the gradient quotient for MetaInit and the loss after one optimizer step for GradInit. AutoInit [2] propagates mean and variance analytically through a Keras model graph and sets each layer's weight scale for unit output variance. A distinct work also named AutoInit, by He, Doshi and Gromov [1], tunes scalar parameter multipliers to make adjacent-block Jacobian norms near one, using Gaussian-vector estimators and a Jacobian log loss. Its diagnostic target overlaps susceptibility propagation; unlike our analytic classifier, it optimizes measured Jacobians on a concrete network. Our external-method table evaluates Bingham and Miikkulainen's analytic AutoInit, not this Jacobian-tuning method. No comparative benefit over the latter is established. These methods produce an initialization; PhaseKit's classifier instead predicts the outcome of a given one, which is why we score it on the networks produced by all of them.

3 Approach

Second-moment propagation on a graph. PhaseKit turns the model into a directed acyclic graph of modules and propagates a scalar second moment q in topological order, starting from the stated input second moment q_0, with default $q_0 = 1$. Standardization does not guarantee this moment for every dataset. A weight layer with measured scale $\hat{\sigma}_w = \mathrm{std}(W)\sqrt{\text{fan-in}}$ maps $q \mapsto \hat{\sigma}_w^2 q + \hat{\sigma}_b^2$ and contributes a factor $\hat{\sigma}_w^2$ to the susceptibility. An activation maps $q \mapsto \mathbb{E}_z[\phi(\sqrt{q}z)^2]$ and contributes $\mathbb{E}_z[\phi'(\sqrt{q}z)^2]$, both computed by numerical quadrature. Pooling, flattening and reshaping nodes pass q through. Residual additions, concatenations and normalization layers have their own rules; the networks studied here contain none of them. The total susceptibility χ_{tot} is the product of the per-node factors along the network. Because q is propagated layer by layer, the check uses the actual trajectory of second moments rather than a fixed point, and can differ from its fixed-point approximation. Predictive superiority over the matched fixed-point decision rule is not established here.

Decision rule. χ_{tot} estimates the factor by which the squared gradient norm changes from the output to the input. PhaseKit predicts the network is trainable (*critical*) when $10^{-4} \leq \chi_{\mathrm{tot}} \leq 10^4$, *ordered*

below the window and *chaotic* above it. Equivalently, the per-layer geometric mean $\bar{\chi} = \chi_{\text{tot}}^{1/L}$ must lie in $[10^{-4/L}, 10^{4/L}]$, so the admissible band around $\chi_1 = 1$ narrows as depth grows. In depth-scale terms the rule is $L\,|\ln \bar{\chi}| \le \ln 10^4 \approx 9.2$, compared with $L|\ln \chi_1| \le 6$ for the 6ξ rule; the two differ in the constant and in where χ_1 is evaluated. The window is recorded in repository commit `2c2d1f5`, preceding the CIFAR-10 study, and is held fixed throughout this evaluation.

4 Implementation

PhaseKit is a Python package built on PyTorch [9]. The graph is obtained with `torch.fx` symbolic tracing, with a fallback that walks the module tree when tracing fails. Weight scales are read from the model's parameters, so the check reflects whatever initialization the model currently holds, including data-dependent ones. The entry point is a single call, `classify(model, input_shape, input_variance=1.0)`, which returns the phase, χ_{tot} and per-node diagnostics. On the networks of §5.2 it takes a median of 0.10 s (maximum 1.26 s) on four CPU cores, including a forward pass on random inputs that records empirical variances for diagnostics. The package also provides `initialize(model)`, which redraws each weight layer with the edge-of-chaos σ_w for the detected activation, and a higher-level `fix(model)`; both appear in §5.3.

5 Evaluation

We ask three questions. **RQ1**: does PhaseKit's predicted phase match whether a plain network trains on CIFAR-10? **RQ2**: how does it compare with other predictors that are available before training? **RQ3**: across initialization schemes, including data-dependent ones, does the phase of the initialized network still predict its outcome, and how do the schemes compare?

5.1 Setup

Data. CIFAR-10 [7] (torchvision copy, MD5-verified on download), standardized per channel. A fixed split of the 50,000 training images gives 45,000 for training and 5,000 for validation; the 10,000 test images are used only for reporting. There is no augmentation.

Networks. MLPs with width 256 and $L \in \{4, 10, 20, 40\}$ weight layers (readout included), activations tanh, ReLU and GELU. CNNs with 32 channels, a stride-2 first convolution, 3×3 convolutions, global average pooling and a linear readout, $L \in \{4, 10, 20\}$, activations tanh and ReLU. No normalization, no skip connections, no dropout. All weights are drawn from $\mathcal{N}(0, \sigma_w^2/\text{fan-in})$ with $\sigma_w \in \{0.5, 0.75, 1, 1.25, 1.5, 1.75, 2, 2.5, 3\}$ and zero biases. This gives 162 configurations; with seeds 0, 1 and 2, 486 networks.

Training. SGD with momentum 0.9, batch size 256, no weight decay and no gradient clipping, for 5 epochs. Each network is trained with every learning rate in a grid, $\{0.1, 0.01, 0.001\}$ for MLPs and $\{0.01, 0.001\}$ for CNNs, giving 1296 runs. Networks of the same configuration are trained simultaneously on one GPU by stacking their parameters, which is numerically equivalent to independent training on a shared minibatch order (verified by a unit test).

Label. Fixed before the runs: a network is *trainable* if, for at least one learning rate, its loss stayed finite throughout and its final validation accuracy is at least 0.20 (chance is 0.10). We also report the thresholds 0.15 and 0.30. For the analysis of failure modes, an untrainable network is called *unstable* if every learning rate produced a non-finite loss or its initial validation loss exceeds $2 \ln 10$, and *stuck* otherwise. Under this label 71.0% of the networks are trainable, and the three seeds of a configuration disagree in only 4 of 162 configurations.

Predictors. All predictors see the exact network that is trained. (i) PhaseKit `classify`, trainable iff the phase is critical. (ii) The depth-scale rule [14]: trainable iff $L \le 6\xi$ with $\xi = 1/|\ln \chi_1(q^*)|$ at the fixed point of the variance map reached from $q_0 = 1$ (our implementation). (iii) A simple first/last-layer gradient-norm

Table 1: Predicting CIFAR-10 trainability of 486 plain networks (label threshold 0.20). Balanced accuracy is the mean of the recall on trainable and on untrainable networks; 95% intervals from 2,000 bootstrap resamples of configurations. "Tuned" is the best balanced accuracy over all thresholds on the continuous score, chosen on the evaluation data itself (an optimistic bound).

Predictor	Bal. acc. [95% CI]	Acc.	Rec. (train.)	Rec. (untrain.)	AUC	Tuned
PhaseKit `classify`	**0.907** [0.865, 0.945]	**0.877**	0.835	0.979	0.984	0.947
Depth scale, $L \leq 6\xi$	0.830 [0.787, 0.873]	0.759	0.661	1.000	0.986	0.942
Per-layer bounds (`classify_ng`)†	0.681 [0.610, 0.750]	0.611	0.450	0.912	0.989	0.957
First/last gradient ratio (ours)	0.515 [0.480, 0.556]	0.702	0.959	0.071	0.565	0.629
Always trainable	0.500	0.710	1.000	0.000	–	–

† MLPs only (324 networks).

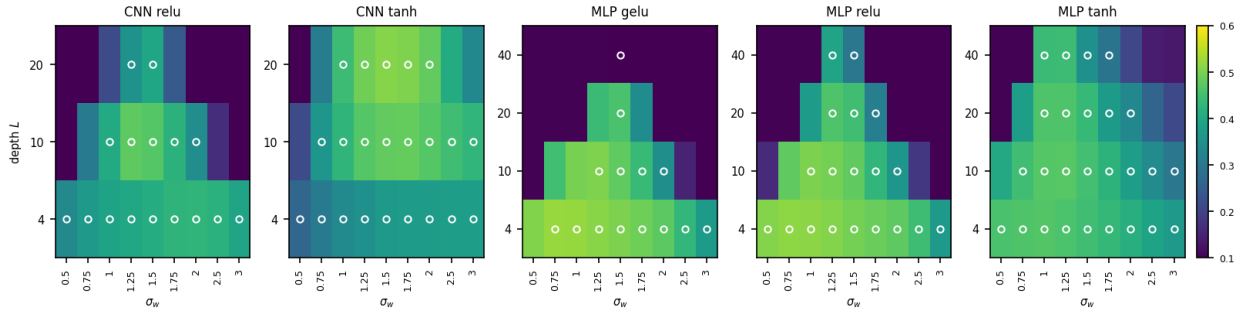


Figure 1: Best final validation accuracy over the learning-rate grid (colour, mean of three seeds) for each depth and weight scale. Circles mark configurations that PhaseKit predicts trainable (majority of seeds).

ratio probe (ours): on one real minibatch of 256 training images, the ratio r of the first- to the last-layer weight-gradient norms; trainable iff the loss is finite and $r^2 \in [10^{-4}, 10^4]$, the same window as (i). (iv) A per-layer variant that thresholds a kurtosis-corrected per-layer susceptibility at fixed, depth-independent bounds (the `classify_ng` function of the same package). (v) The constant predictor "always trainable". For predictors with a continuous output we also report the area under the ROC curve (AUC).

Compute. All training ran on single NVIDIA L4 GPUs through Modal; predictors ran on CPU containers. Configurations, raw per-run logs (loss, validation and test accuracy per epoch) and the exact commands are in the repository.

5.2 RQ1 and RQ2: predicted phase versus measured trainability

Table 1 and Figure 1 give the results. PhaseKit's prediction agrees with the measured outcome for 288 trainable and 138 untrainable networks and disagrees for 3 + 57, a balanced accuracy of 0.907. Its errors are asymmetric: 99.0% of the networks it calls trainable do train, and the other 57 errors are trainable networks just outside the predicted window. Of these 57 false negatives, 47 lie at most one step of the σ_w grid from a weight scale predicted trainable at the same depth and the other 10 lie two steps away (Figure 1). The result is stable across architectures and activations, with balanced accuracy 0.924 (MLP, tanh), 0.938 (MLP, ReLU), 0.826 (MLP, GELU), 0.923 (CNN, tanh) and 0.938 (CNN, ReLU), and across label thresholds: 0.898 at 0.15 and 0.897 at 0.30.

An independent paired bootstrap groups all weight scales and seeds within each of 18 architecture/activation/depth combinations. Across 2,000 resamples, the balanced-accuracy gain over the fixed 6ξ rule has a 95% interval of $[0.043, 0.109]$ (artifact `study_audit/reaggregate.py`). This interval conditions on the evaluated architecture grid. A matched-window control replaces the classical constant 6 by the same fixed $\ln 10^4$ used by PhaseKit, without tuning on the evaluation labels. It reaches 0.913 balanced accuracy; the paired PhaseKit minus control interval is $[-0.029, 0.015]$. Window calibration materially affects the comparison.

On their evaluated subsets, the three mean-field scores rank networks almost identically (AUC between 0.984 and 0.989); they differ in the decision rule. With a threshold chosen in hindsight on the evaluation data, each would reach a balanced accuracy between 0.942 and 0.957. PhaseKit's fixed window reaches 0.907, within 0.04 of the hindsight optimum for its own score. The constant 6 of the depth-scale rule is conservative on this task. The rule never calls an untrainable network trainable, but it rejects a third of the trainable ones (recall 0.661). Thresholding a per-layer susceptibility at depth-independent bounds misplaces the boundary in both directions. It rejects 60 trainable networks at depth 4 and accepts 10 untrainable ones at depth 40, because the admissible per-layer band must narrow as depth grows. Our first/last-layer gradient ratio probe is close to chance (0.515) even though it uses real data and a backward pass; this says nothing about data-based diagnostics in general, only about this two-layer ratio. The weight gradient of a layer is the product of the backpropagated error and the layer's input; in the ordered phase the error shrinks toward the input while the forward activations shrink toward the output, and for ReLU networks the two factors cancel exactly in the first-to-last ratio. A two-layer ratio therefore does not see the depth-wise product that determines trainability.

The phase also predicts how training fails. Of the networks predicted ordered, 63 fail by stalling with a finite loss and 0 diverge; of the networks predicted chaotic, 68 fail by divergence or saturated initial logits and 7 stall. In practice, an ordered prediction calls for a larger weight scale and a chaotic one for a smaller scale or learning rate.

5.3 RQ3: initialization schemes

Setup. We take eight deep plain networks from the E1 families (MLPs with tanh, ReLU and GELU at $L = 20, 40$; CNNs with tanh and ReLU at $L = 20$) and initialize each with seven external schemes: PyTorch's default `reset_parameters`, Xavier-normal [4], He-normal [6], LSUV [8], GradInit [17], MetaInit [3], and AutoInit [2]. LSUV uses the authors' PyTorch code with an orthogonal start and unit-variance target on one batch of 256 training images. GradInit and MetaInit use the GradInit repository at a pinned commit, its no-BatchNorm VGG-19 hyperparameters (390 iterations, scale learning rate 10^{-2}, minimum scale 0.01, gradient clip 1), and its Kaiming-normal start. AutoInit runs in the authors' TensorFlow package on an equivalent Keras model whose weights are copied into the PyTorch network. Each network is trained for 10 epochs with the E1 optimizer and learning-rate grid; the best final validation accuracy determines the selected learning rate per seed. Before training, PhaseKit `classify` is run on each initialized network.

Phase prediction across external initializers. Across 168 initialized networks, all 108 classified critical reach the trainability threshold. Balanced accuracy is 0.912; an independent bootstrap over eight architecture clusters gives a 95% interval $[0.847, 0.964]$, conditional on these architectures. These results include weights produced by data-dependent scaling and AutoInit's analytic propagation, extending the E1 evaluation beyond one Gaussian initialization scale. The 23 false negatives are classified ordered but reach 20% validation accuracy; the measured criterion remains tied to the optimizer and finite training schedule. Per-architecture training logs and selected learning rates are included in the artifact.

Input second moment diagnostic. We hold one initialized 20-layer tanh MLP fixed and compute an autograd input/output squared gradient-norm ratio on 32 real CIFAR-10 training images, using fixed Gaussian output cotangents. Input scaling by $0.01, 1, 100$ yields ratios $21725, 9.176, 0.0812$, while the default data-free calculation reports $\chi_{\text{tot}} = 9.193$ for each. Passing the known preprocessing second moment $q_0 = s^2$ through the API yields predictions $16473, 9.193, 0.0640$ and changes the low-input-scale phase to chaotic. This diagnostic supports exposing the input second moment in the interface; it measures gradients on one fixed model and minibatch rather than training outcomes (artifact `study_audit/input_scale_stress.py`).

Independent-weight boundary replication. A prospectively frozen synthetic diagnostic uses six fresh weight seeds in a 20-layer tanh MLP (input dimension 128, width 64), Gaussian weights with scale 1.5 and zero biases. Each complete parameter hash is distinct. Across three Gaussian-input scales and four output cotangents per cell, the median absolute $\log_{10}$ error between the predicted gradient factor and the cotangent-averaged autograd ratio is 2.276 with the default moment and 0.097 with the measured input moment. All

six low-scale cells have empirical ratios above 10^4: the default prediction calls them critical, while the stated-moment prediction calls them chaotic. This replicates the input-assumption boundary under fresh weights; it does not validate training efficacy or generalization (artifact `study_audit/independent_moment.py`).

6 Discussion and Limitations

The evaluation covers plain feed-forward MLPs and CNNs on one image dataset, trained with SGD with momentum for short schedules. Longer training can rescue networks near the boundary, so the measured trainable region would widen with more epochs, and adaptive optimizers rescale gradients per parameter and are less sensitive to vanishing gradients; we did not evaluate either. The data-free prediction assumes an input second moment of one, as approximated by the study preprocessing. The API also accepts a stated input second moment. Widths are 256 (MLP) and 32 channels (CNN); the mean-field quantities are width-independent to first order, but much narrower networks have larger fluctuations around them. The networks studied contain no normalization and no residual connections. PhaseKit handles those through separate propagation rules, which this paper does not evaluate; for such architectures trainability is less sensitive to the weight scale and a phase prediction is less informative. The matched classical-window point estimate exceeds ours, with a paired cluster interval including zero; no predictive advantage is established. There is no held-out dataset or decision-benefit study, and own-initializer training across independent weight seeds remains incomplete. Finally, the label requires a fixed accuracy after a fixed budget; we report three thresholds, and the conclusions do not change between them.

7 Conclusion

A mean-field susceptibility computed from an untrained `nn.Module`, with no data and in about a tenth of a second, predicts whether a plain deep network will train on CIFAR-10 with balanced accuracy 0.907, under a selected short-budget label. PhaseKit packages this check for traced PyTorch models. It provides early warnings about initialization under the evaluated protocol, and it can check the output of other initialization procedures.

References

[1] T. He, D. Doshi and A. Gromov. AutoInit: Automatic initialization via Jacobian tuning. arXiv:2206.13568, 2022.

[2] G. Bingham and R. Miikkulainen. AutoInit: Analytic signal-preserving weight initialization for neural networks. arXiv:2109.08958, 2021.

[3] Y. N. Dauphin and S. S. Schoenholz. MetaInit: Initializing learning by learning to initialize. In *Advances in Neural Information Processing Systems 32*, 2019.

[4] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, PMLR 9:249–256, 2010.

[5] B. Hanin. Which neural net architectures give rise to exploding and vanishing gradients? arXiv:1801.03744, 2018.

[6] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. arXiv:1502.01852, 2015.

[7] A. Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[8] D. Mishkin and J. Matas. All you need is a good init. arXiv:1511.06422, 2015.

[9] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. arXiv:1912.01703, 2019.

[10] J. Pennington, S. S. Schoenholz, and S. Ganguli. Resurrecting the sigmoid in deep learning through dynamical isometry: theory and practice. arXiv:1711.04735, 2017.

[11] B. Poole, S. Lahiri, M. Raghu, J. Sohl-Dickstein, and S. Ganguli. Exponential expressivity in deep neural networks through transient chaos. arXiv:1606.05340, 2016.

[12] D. A. Roberts, S. Yaida, and B. Hanin. The principles of deep learning theory. arXiv:2106.10165, 2021.

[13] A. M. Saxe, J. L. McClelland, and S. Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. arXiv:1312.6120, 2013.

[14] S. S. Schoenholz, J. Gilmer, S. Ganguli, and J. Sohl-Dickstein. Deep information propagation. arXiv:1611.01232, 2016.

[15] L. Xiao, Y. Bahri, J. Sohl-Dickstein, S. S. Schoenholz, and J. Pennington. Dynamical isometry and a mean field theory of CNNs: How to train 10,000-layer vanilla convolutional neural networks. arXiv:1806.05393, 2018.

[16] G. Yang and S. S. Schoenholz. Mean field residual networks: On the edge of chaos. arXiv:1712.08969, 2017.

[17] C. Zhu, R. Ni, Z. Xu, K. Kong, W. R. Huang, and T. Goldstein. GradInit: Learning to initialize neural networks for stable and efficient training. arXiv:2102.08098, 2021.